

PS 2: Part I

Alan Xu

Problem 1: Sorting practice

1-1) selection sort after 2 passes

3 4 18 24 33 40 8 10 12

1-2) insertion sort after 4 passes, the first $n+1$ elements are sorted

4 10 18 24 33 40 8 3 12

1-3) shell sort, initial pass, increment 3

8 3 4 10 18 12 24 33 40

(Here, we can group each set of numbers that are %increment, in this case 3, away, and sort them, and place them back into their k , $k+incr$, $k+2*incr$ spots)

1-4) bubble sort, 3rd pass

4 10 18 8 3 12 24 33 40

1-5) quicksort, initial partition, return value of j is 7

10 18 4 24 12 3 8 40 33

1-6) radix sort, initial pass, ones digit, otherwise kept in place
{10,40}, {12}, {33, 3}, {4, 24}, {18, 8}

1-7) After 4 calls to merge, we have merged (indices)

- a. [0] and [1]
- b. [0,1] and [2]
- c. [3] and [4]
- d. [0,1,2] and [3,4].

so we expect the subarray [0:4] to be sorted

4 10 18 24 33 40 8 3 12

Problem 2: Counting comparisons

2-1) Insertion sort only makes **4** comparisons. We check $\text{arr}[i] < \text{arr}[i-1]$ for $i = 1$ to $i = 4$. The comparisons inside to place $\text{arr}[i]$ into the right place are passed over because we never meet the condition initial if condition.

2-2) Also 10 comparisons, we check at every step in bubblesort, although the swap may not happen.

2-3)

- a. The first partition does **6** comparisons (indices 0 to 2, 1 to 2, then 2 to 2; then from the other side, 4 to 2, 3 to 2, 2 to 2). Now our split value is 2, so we partition into [0:2] and [3:4]
- b. The left hand partition [0:2] makes **4** comparisons (same logic as before). Split value is 1, so we have [0:1] and [2]
- c. The [0:1] array has pivot index 0. We make **1** comparison for i and exit the do-while. We make **2** comparisons for j (one at index 1, one at index 0) and return $j = 0$. This is at base cases, so no more partitions
- d. The [2] array is at base case, so no partition and **0** comparisons
- e. The [3,4] array has pivot index 3, we make **3** comparisons like in part c.

This is a total of **16** array element comparisons.

Problem 3: Comparing two algorithms

3-1) This is $O(n \log n)$. Merge sort is $O(n \log n)$. The subsequent for loop runs once (n steps) and makes one comparison each (constant time), so its $O(n)$ runtime is dominated by the merge sort at the beginning.

3-2) This is $O(n^2)$. Consider an array with no duplicates, so we never break out of the inner loop. The outer loop runs over all n items in the array, then the inner loop runs $n-1, n-2, \dots$ times. This ends up being around $n(n-1)/2$ which was $O(n^2)$.

Problem 4: Sum generator

4-1) $n * (n+1) / 2$, or $n^2/2 + n/2$

4-2) This is $O(n^2)$. As n gets large, our expression is dominated by $n^2 / 2$ (so we can ignore the $n/2$). In big O notation, we also drop out any constant scalars, so the final answer is $O(n^2)$

Problem 5: Improving generateSums()

required for grad-credit; optional for others

5-1)

```
public static void generateSums(int n){
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
        System.out.println(sum);
    }
}
```

5-2) This is $O(n)$. The loop runs n times, and there are a constant number of operations within each run of the loop.

Problem 6: Practice with references

6-1)

Expression	Address	Value
n	0x100	0x712
n.ch	0x712	'n'
n.prev	0x718	0x064
n.prev.prev	0x06A	0x360
n.prev.next.next	0x714	null
n.prev.prev.next	0x362	0x064

6-2)

```
// assign the data for the inserted node first
x.prev = n.prev;
x.next = n;
// then change the relations of the existing nodes
n.prev.next = x;
n.prev = x;
```